OOP Group 9 Term Project Report

Object Oriented Programming (COMP0213)

Aayan Islam | 24102520
Helitha Cooray | 24161798

December 10, 2025

Contents

Code Submission

The code for this project can be found in the following GitHub Repository **(Only the main branch)**.A full guide on how to get the code to work has been provided in the **README.md** file.

 **github.com/ItzSmudge/COMP0213_Group9_GraspPlanning**

1 Introduction

For this object-oriented programming project, we were given the task of using various grippers to attempt to grasp a variety of objects from random initial positions and orientations. This was done within the pybullet environment which is used for simulating robotic systems.

This process is done by creating a set of random positions and orientation relative to the object we are grabbing. There is additional noise added to ensure realism. Depending on whether the gripper can hold the object long enough, the grasps are labeled as a success or a failure. The overall goal is to generate training data that can be used to train a machine learning model such that it is able to predict the probability of a successful grasp based on position and orientation. To understand the effectiveness of the classifier models, different classifiers were used and examined to find which one is the most accurate.

1.1 Class Structure and OOP Principles

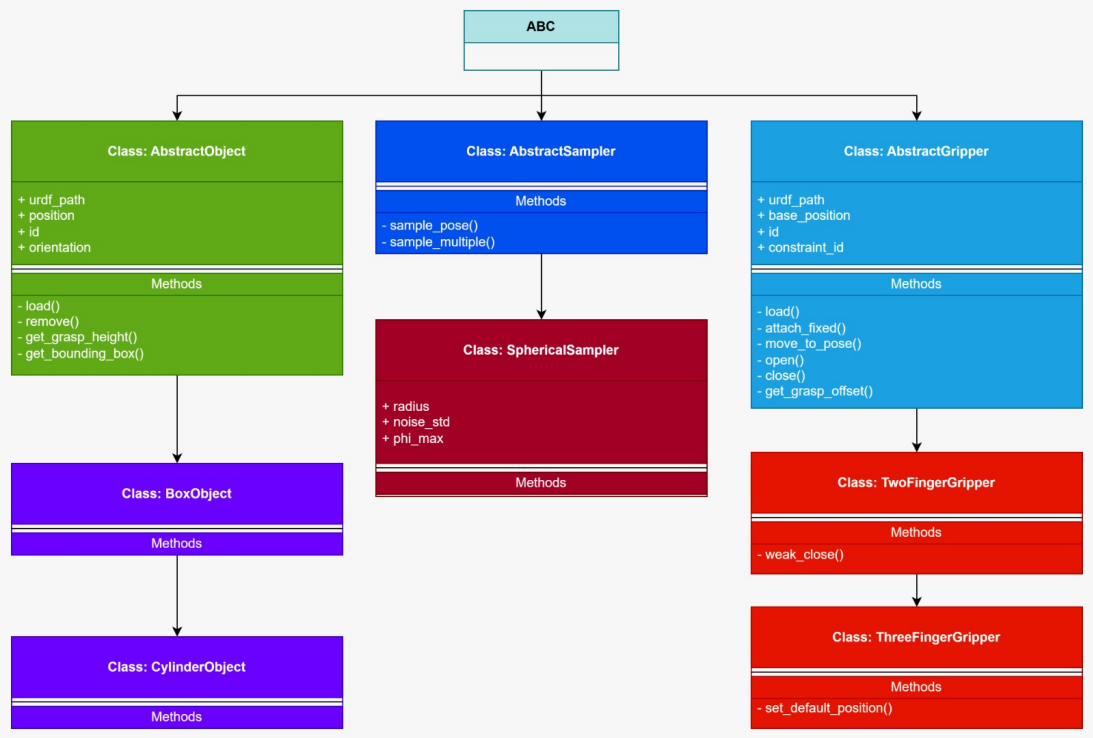
Figure 1 showcases the classes used for this project. The **ThreeFingerGripper** and the **TwoFingerGripper** are both child classes of the **AbstractGripper**. This allows us to easily create new grippers and add them on as new child classes . The objects used are also child classes from **AbstractObject**. The **SphericalSampler** handles all the grasping position and orientation generation. We make use of abstract classes as they enable abstract methods and properties to be defined which ensures every function has the required attributes and functions.

Additionally, the **GraspSimulator** handles all the pybullet setups as well as actually run the simulations. It connects the pbullet engine to the simulation and executes the grasping as well as checking for success. The **GraspPipeline** handles all the data as well as running the entire pipeline. **GraspClassifier** is used with all the classifier models which allows for tuning, training and predicting. The **DataManager** is used for the data. It adds the randomly generated position, orientation and outcome to the csv as well as reading and balancing the datasets

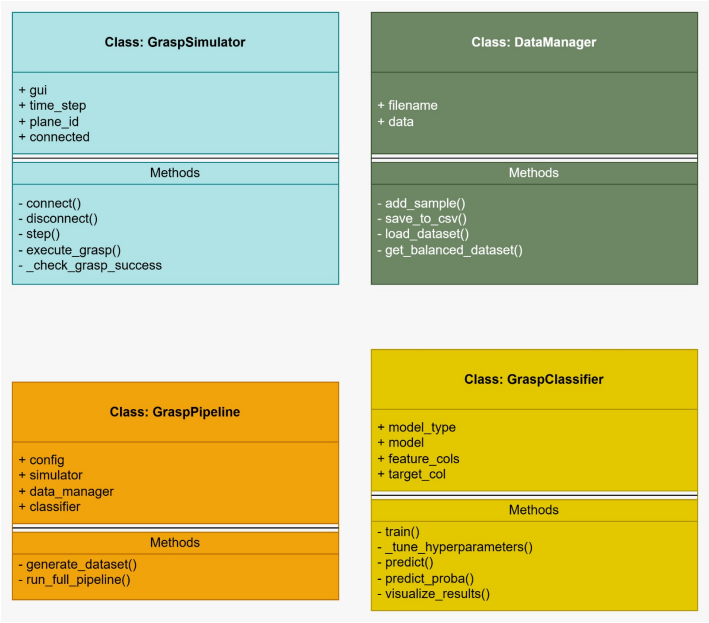
Many OOP principles were used throughout this project.

- **Inheritance**: Parent classes were developed to contain the attributes and methods that were needed across all children classes. This meant that every child class instantiated would inherit the functions such as **open** and **load** which were needed across all instances. This also meant that code was not re-written hence making it much more modular and readable.
- **Abstraction**: Abstract methods allowed us to create functions that contained empty methods. This meant that if the child class did not contain those methods, errors would be returned. This was key for the grippers as it ensured we had **open** and **close** methods for each individual gripper. These methods needed to be different as the gripper themselves had different joints and therefore needed slight changes in the code.

- **Polymorphism**: Each gripper and object had fundamental methods that were the same across all of them yet had slight differences. The use of polymorphism allowed us to use the same functions for different grippers to achieve the same desired output but with minor changes



(a) Abstract class tree



(b) Other classes

Figure 1: UML Diagram

1.2 Libraries Used

Pybullet was used to simulate the enviroment. The physics simulations were from pybullet. **Pandas** was for data handling. It allowed us to easily add grasp data as well as read it for

classification. **Numpy** was used for calculations. Numpy uses vectorized methods which are much faster and efficient making the simulation faster. **Scikit-Learn** was used for all the classifiers. We accessed each classifier and used the inbuilt train and predict functions. We were also able to evaulate the models accuracy using this library. **Matplotlib** and **Seaborn** were used to plot to allow us to visually see and represent the data. This enabled us to easily see how well our models performed.

The machine learning workflow was done using several packages in scikit-learn. Multiple utility functions were used such as test_train_split and StandardScalar, which allowed for data separation and feature normalization. GridSearchCV allowed us to select the best hyperparameters. The actual models themselves consisted of LogisticRegression and SVC, ensemble and tree-based models such as RandomForestClassifier and the high-performance GradientBoostingClassifier, and a neural network classifier, MLPClassifier.

2 Dataset

2.1 Pipeline Description

Our dataset creation pipeline follows a systematic three-phase approach to generate our grasp training data. The complete workflow includes events from initial pose sampling through simulating the grasp physically in PyBullet to the construction of the datasets.

We generated the sample grasp positions using **spherical coordinate sampling**. This was done by constructing a semi-spherical path around the generated object, along which the initial position of the gripper can be randomly chosen, so it always remains centered on the target object. **Why spherical sampling:** We went with this method because it would effectively distribute the grasp attempts around the object while maintaining a constant approach distance. The addition of noise gives realistic variations, and simulates uncertainty and sensor errors that can occur in real robotic systems.

Sampling Parameters

- **Sphere base radius**: $0.4\,\mathrm{m}$
- **Positional noise**: Gaussian noise with $\sigma = 0.003\,\mathrm{m}$ added to the radius.
- **Polar angle** φ: Constrained to $[0°, 108°]$ to focus on top-hemisphere approaches and ensure the gripper does not start from below the simulation plane.
- **Azimuthal angle** θ: Provides full 360° coverage around the gripper from above the plane.
- **Roll variation**: ± 0.2 radians to introduce variations in rotation.

Object-Specific Adjustments

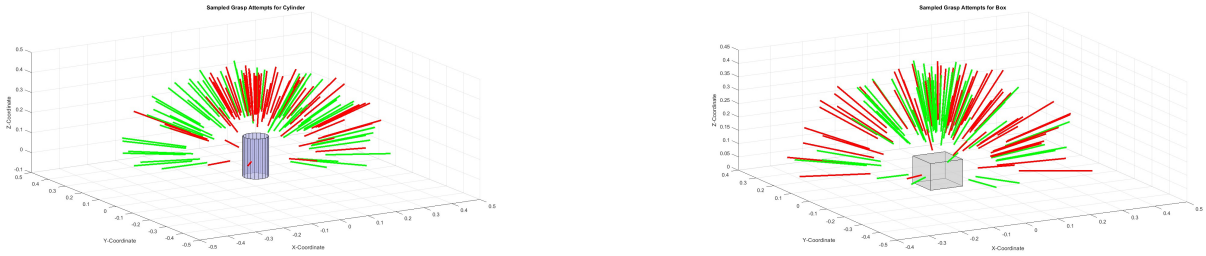
Different gripper-object combinations require orientation corrections to account for mechanical geometry:

- **Two-finger + Cylinder**: Pitch adjusted by $-90°$ to align parallel fingers with the cylinder's longitudinal axis.
- **Three-finger + Box**: Pitch adjusted by $+90°$ to accommodate the radial finger arrangement.

The sampling algorithm works by first generating random spherical coordinates (radius, polar angles, azimuthal angle), then converting them into Cartesian positions. The gripper orientation is calculated to point towards the centre of the object, ensuring the gripper approaches from the sampled directions.

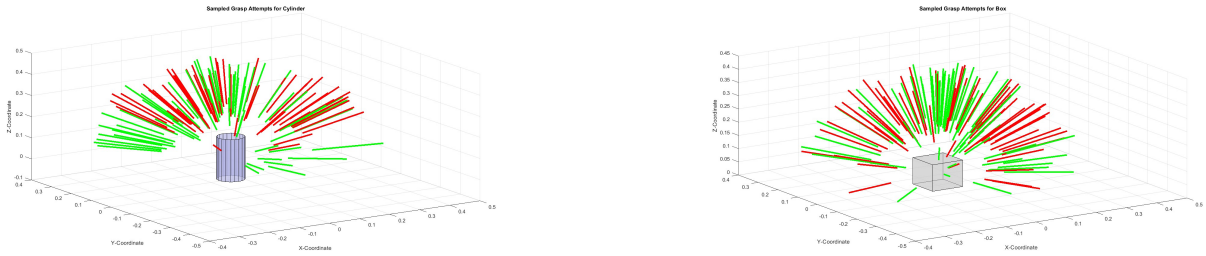
2.2 Specialized Datasets

Rather than training a single classifier, we generate four independent CSV files; one for each gripper-object combination. This allows each model to learn combination-specific grasp dynamics without interference from other configurations. Furthermore, each gripper-object pair has unique geometric constraints and success patterns. For example, the two-finger+cylinder requires precise lateral alignment (hence the -90° pitch correction), while three-finger+box benefits from distributed contact points. Training separate classifiers allows each model to specialize in its specific configuration without being confused by data from incompatible combinations.



(a) Two Finger gripper with the cylinder (b) Two finger gripper with the box

Figure 2: Two finger gripper random grasps



(a) Three Finger gripper with the cylinder (b) Three finger gripper with the box

Figure 3: Three finger gripper random grasps

2.3 Labelling and Preprocessing

Grasps are evaluated as successful **(1)** or unsuccessful **(0)**, using height-based thresholds with temporal stability verification, as some grasps might end up with the object slipping after remaining stationary upon lifting., which we particularity noticed with the cylinder object seeming to be more "unstable" compared to the box.

- **Box threshold**: 0.25 m after 30-step hold.
- **Cylinder threshold**: 0.20 m after 50-step hold

therefore capturing the following situations which are considered as grasp failures:

- **Immediate drop**: Object falls during approach/closure.

- **Slip during lift**: Object slides out during vertical motion.
- **Delayed release**: Object falls during hold period.
- **Toppling**: Object tilts or rotates and loses stable contact.

2.4 Dataset Construction

The datasets were created with the following columns:

Table 1: Grasp Dataset Features

Feature	Type	Range	Description
x, y, z	float	$[-0.4, 0.6]$ m	3D grasp position in world coordinates
roll, pitch, yaw	float	$[-\pi, \pi]$ rad	Gripper orientation (Euler angles)
success	binary	$\{0, 1\}$	Grasp outcome label
gripper	string	-	Metadata: gripper type
object	string	-	Metadata: object type

3 Classification

3.1 Classifier Setup and Evaluation

To ensure that we have the highest validation and testing accuracy as possible, five classification models were tested: logistic regression, Support Vector Machine (SVM), random forest, gradient boosting and a neural network (Multi-Layer Perception). Each classifier was implemented using scikit-learn with preprocessing where appropriate.

The dataset for each gripper+object combination was split into training (80%) and validation (20%) to ensure that ample data was used for training.

Each model was initialized using typical default parameter values. To further improve performance, we ran **hyperparameter tuning** using **GridSearchCV** from scikit-learn, performing a **3-fold cross validation** over a grid of parameters which we predefined for each model:

- **Logistic Regression:** Regularization strength $C \in \{0.1, 1.0, 10.0\}$
- **SVM (Support Vector Machine):** Regularization $C \in \{0.1, 1.0, 10.0\}$, kernel coefficient $\gamma \in \{\text{scale}, \text{auto}\}$
- **Random Forest:** Number of trees $\in \{100, 200, 300\}$, maximum depth $\in \{8, 12, 16\}$
- **Gradient Boosting:** Number of estimators $\in \{100, 150, 200\}$, learning rate $\in \{0.05, 0.1, 0.15\}$
- **Neural Network:** Hidden layer sizes $\in \{(32, 16), (64, 32), (128, 64)\}$, regularization $\alpha \in \{0.0001, 0.001\}$

The best model parameters were selected based on cross-validation accuracy. The optimization process ensures that the final classifier used the most effective settings discovered during the grid search process. Furthermore our class weights were set to handle any **class imbalance** in the training data. We noticed the effect of the data imbalance for all four classifiers with our data, as our datasets had a **significantly greater percentage** of successful grasps. This resulted in all the classifiers doing well when it came to predicting the **True Positive** outcomes. While developing the pipeline and the sampler, we accumulated data over a large number of

tests, which for some datasets included a greater proportion of failed attempts, while some were more balanced. We noticed the difference in the predictions when this data was used with our final classification pipeline.

3.2 Fixed Training Set Evaluation

After selecting the best classifier parameters from the training process, and selecting the best model for each dataset, the trained model performance was evaluated on completely new test grasps generated through the PyBullet simulation. The trained classifier predicted success or failure for each pose before execution and these predictions were compared against the actual simulation result for each grasp.

Table 2: Performance on 50 newly generated test grasps

Gripper-Object Pair	Test Accuracy	Actual Success Rate	Predicted Success Rate
TwoFinger + Box	89.87%	82.45%	90.00%
TwoFinger + Cylinder	100.00%	75.24%	78.00%
ThreeFinger + Box	75.17%	77.15%	78.00%
ThreeFinger + Cylinder	82.19%	77.06%	72.00%

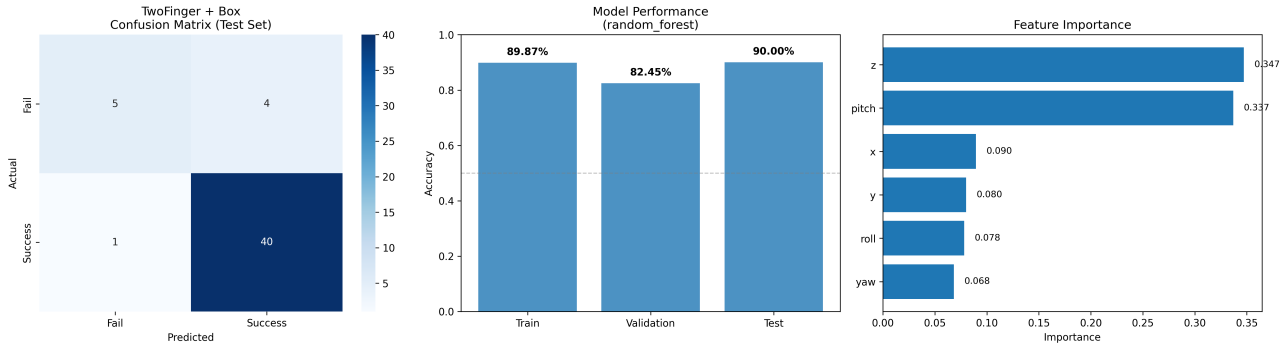


Figure 4: Results for Two finger gripper and box

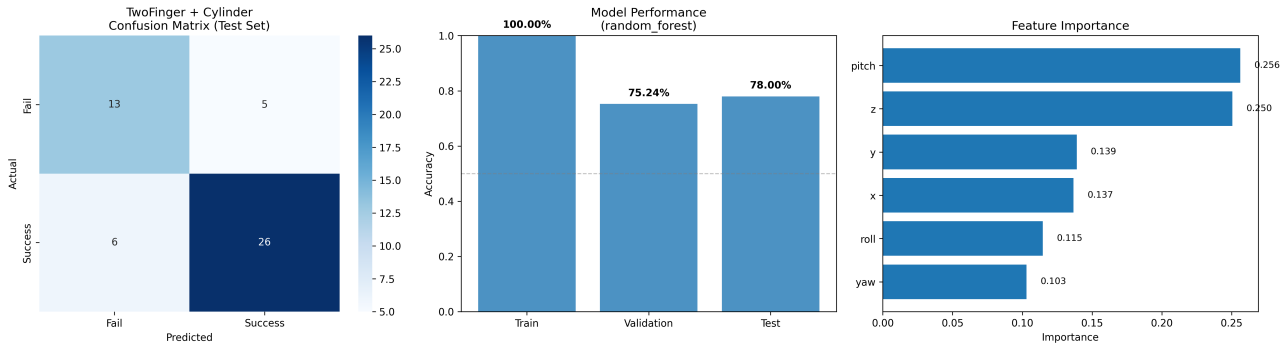


Figure 5: Results for Two finger gripper and cylinder

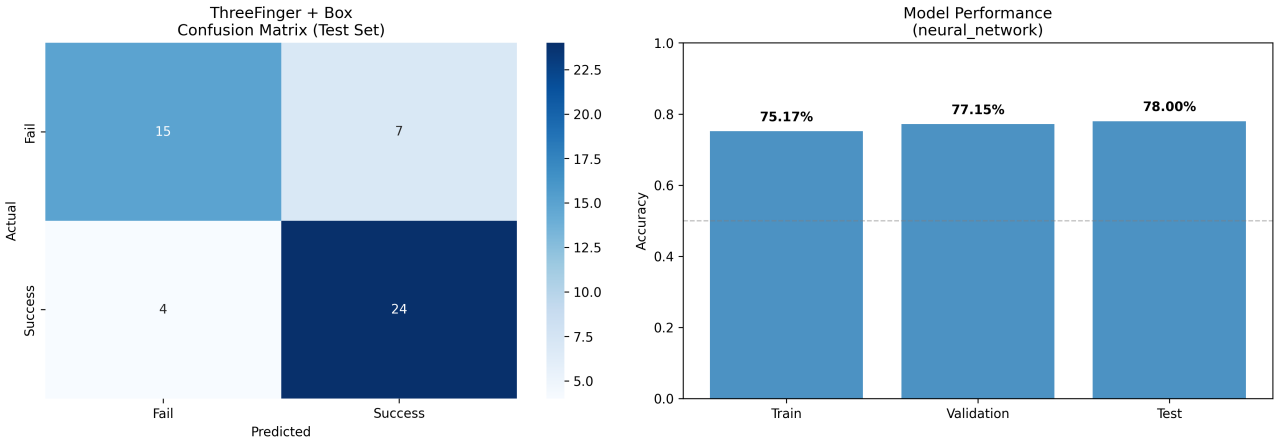


Figure 6: Results for Three finger gripper and box

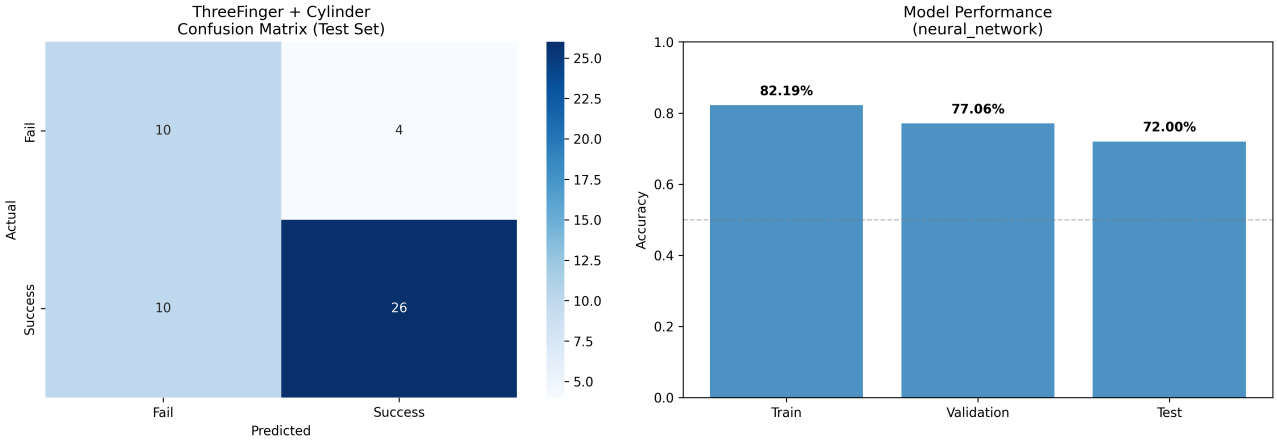


Figure 7: Results for Three finger gripper and cylinder

4 Discussion and Conclusion

Achievements

Overall, the main achievement of this project was that we successfully created a grasping simulation that was spawned by grippers of various types and moved to different objects to grab and pick them up. Additionally, we also created four datasets with each one having a specific gripper and object. This meant that when classifying, we could guarantee the most accurate training data to the model. By separating the data, we could further prevent bias since other shapes and grippers would not influence the specific data of the gripper and object being used. We also thoroughly trained our model with over 1000 samples. This proved to be beneficial as it provided us with high validation and test accuracies of around 70%. A more impressive achievement was that with the 4 separate datasets, we managed to get 92% test accuracy during one of our tests.

This project served as a comprehensive platform for integrating Object-Oriented Programming principles with advanced data science toolkits. By using custom classes, we achieved significant gains in code modularity, reusability, and maintainability, effectively decomposing the overall application into manageable sections. Furthermore, hands-on experience with NumPy and Pandas has greatly enhanced our capability for data handling and efficient manipulation of

large datasets. Overall this has given us insight into the practical application and evaluation of various Scikit-learn classification models has solidified our foundational understanding of machine learning models. For classification models that required models such as random forest, we implemented an additional feature of visualizing the importance of the features of the samples, to show how each feature affected the prediction outcome.

Weaknesses

A weakness was definitely our failure criterion. It was very hard to gauge and find a fitting range to consider grasps true successes and failure. A definite failure can be defined as the object staying on the floor at the end of the grasp. However, when the object was being grasped, we noticed it moving around in the gripper. This can be accounted to a multitude of reasons ranging from the gripper strength being too high to the simulation not being able to compute the exact state of the object, causing it to move around in the gripper.

Another weakness of this project was the low initial success rate. When starting the classification section, we noticed lots of incorrect predictions. This was due to using one large dataset which caused bias and lead to misclassifications when testing other gripper+object combinations. To approach this, we instead split our data in four separate datasets, one for each gripper+object combination. This improved our accuracy results by a large margin which made our confusion matrices look much more appealing.

Additionally, we found how the imbalance of the number of successful and unsuccessful grasp can affect the classifier predictions. For all four of our datasets, we managed to generate grasp samples, from which **over 50%** were successful. Despite creating balanced datasets for training, this affected the prediction outcomes, as we see in the confusion matrices, where the **True Positive (TP)** (bottom left) predictions were made very well, unlike the **True Negative (TN)**

Challenges

- **URDF paths:** Since many of the shapes in pybullet we already included, there were not many issues. However, for the cylinder specifically, the exact path had to be written down for the code to actually find the correct path. Even if the cylinder URDF file and the code were in the same folder, the code would fail to find the file. Additionally, there were two different cylinder variants which we were unaware of, leading to each of us using different cylinder which had lead to us generating different results.
- **Three-finger gripper:** The three-finger gripper was surprisingly efficient at grabbing objects. However, this was counter-productive as nearly all grasps were being registered as successes. This made it difficult to have enough failures to balance out the training. The solution to this was implementing more constraints such as slipping and movement during the grasp.